Practice Assignment 2: Device Management Software

.NET Core 10.0 MVC • Entity Framework Core • Clean Code • SQLite

Student: Nguyen Duc Anh
Student ID: 2031200076

Date: October 2026

1. Project Overview & Architecture

The **Device Management Software** is built using **ASP.NET Core 10.0 MVC** with **Entity Framework Core** and **SQLite**. The architecture implements Clean Code principles and Separation of Concerns (SoC) across distinct layers:

Folder / Component	Key Classes & Files	Architectural Responsibility
Models/	Device, DeviceCategory, User, DeviceStatus	Domain entities with DataAnnotations validation & enum definition.
ViewModels/	DeviceFilterViewModel, DashboardViewModel	Encapsulates search queries, select lists, and dashboard metrics.
Data/	AppDbContext, DbInitializer	EF Core Code-First DbContext with unique constraints & data seeding.
Controllers/	Home, Categories, Devices, Users	Async controllers handling CRUD, search/filter, and business rules.
Views/	Razor Views + Bootstrap 5 + Icons	Responsive UI, search toolbars, status badges, and feedback alerts.

2. Created Classes & Domain Model

DeviceCategory • Id (int, PK) • Name (string, Required, Unique Index) • Description (string?) • Devices (ICollection<Device>)	**User (Employee)** • Id (int, PK) • FullName (string, Required, max 100) • Email (string, Unique, EmailAddress) • PhoneNumber (string, Required, Phone) • Devices (ICollection<Device>)
DeviceStatus (Enum) • InUse = 1 (Display: 'In use') • Broken = 2 (Display: 'Broken') • UnderMaintenance = 3 (Display: 'Under maintenance')	**Device** • Id (PK), Name, Code (Unique Index, regex) • CategoryId (FK to Category), Status (Enum) • EntryDate (Date), UserId (Nullable FK to User)

3. Feature Implementation Details

Feature 1: Device Category Management

- **CRUD Operations:** CategoriesController provides full Add, Edit, Delete, and List actions.
- **Duplicate Validation:** Validates uniqueness of category names using EF Core async queries before saving.
- **Data Integrity Protection:** Checks if devices are assigned to a category before deletion; blocks deletion with a friendly alert if devices exist, preventing orphaned records.
- **Display Devices by Category:** Dedicated action CategoriesController.Devices(int id) filters and renders all hardware units belonging to the specified category.

Feature 2: Device Management

- **Attributes Tracked:** Device name, unique device code (e.g. DEV-001), category selection, status (In use, Broken, Under maintenance), date of entry, and assigned employee.
- **Input Validation:** Server-side Model validation + client-side jQuery unobtrusive validation. Enforces unique device code check against the database.
- **Details & Deletion:** Details view displays hardware specifications and user contact; Delete view provides clear confirmation safeguards.

Feature 3: User / Employee Management

- **User Attributes:** Full name, unique email address, phone number, and organizational department.
- **Device Allocation:** Users are associated with devices via a foreign key relation. Users/Details.cshtml lists all devices currently in possession of the selected user.
- **Cascade Protection:** Configured OnDelete(DeleteBehavior.SetNull) in EF Core: deleting an employee releases assigned devices back to unassigned inventory without deleting the equipment.

Feature 4: Search and Filter Data

Implemented dynamically in DevicesController.Index using LINQ query composition:

```
var query = _context.Devices.Include(d => d.Category).Include(d => d.User).AsQueryable();
if (!string.IsNullOrWhiteSpace(searchTerm))
    query = query.Where(d => d.Name.ToLower().Contains(searchTerm.ToLower()) || d.Code.ToLower().Contains
(searchTerm.ToLower()));
if (categoryId.HasValue && categoryId.Value > 0)
    query = query.Where(d => d.CategoryId == categoryId.Value);
if (status.HasValue)
    query = query.Where(d => d.Status == status.Value);
```

- **Search:** Searches simultaneously across both device name and device code.
- **Filter:** Filters simultaneously by Category and Status. Includes an instant Reset filter button.

4. Clean Code Principles & Coding Standards Applied

- **Single Responsibility Principle (SRP):** Each controller is solely responsible for one entity aggregate (Categories, Devices, Users).
- **Don't Repeat Yourself (DRY):** Centralized navigation in _Layout.cshtml, reusable validation partials, and unified ViewModels.
- **Naming Conventions:** Strict adherence to C# standards: PascalCase for classes/methods/properties, camelCase for local variables and parameters.
- **Asynchronous I/O:** All controller actions utilize async/await and EF Core async methods (ToListAsync, AnyAsync, SaveChangesAsync) for high throughput.
- **Defensive Programming:** Graceful error handling, TempData alerts for success/error feedback, and FK integrity constraints.
- **Intuitive UI & UX:** Responsive Bootstrap 5 layout, color-coded status badges, and an interactive Home Dashboard summarizing inventory statistics.

5. Execution Guide & Submission

- **Automatic Database Initialization:** DbInitializer automatically creates the SQLite database (devicemanagement.db) and seeds sample categories, users, and devices upon startup.
- **Run Command:** dotnet run --project DeviceManagement/DeviceManagement.csproj then open http://localhost:5000.
- **Submission Package:** Source code archive ready for Moodle and initialized with Git for public GitHub repository submission.